The types of Deep Learning frameworks you have come across:

Tensorflow : Models usually are saved with .ckpt extensions, you can also use frozen graph (saved model) format .pb.

The main difference is in .ckpt you can load the model and keep training it if you want. Frozen .pb models can only be used for inference.

Pytorch : Models are saved with .pth.

Keras: You can use dictionary .h5 formats for saving model weights.

Example: say I have 3 layers in the model. Layer 1 is named l1cnn, layer 2 is named l2cnn and layer 3 is named l3nn

M[“l1cnn”]=tensorA

How will I choose a framework?

a. Do I want to write from scratch, then I can choose any framework I am comfortable in
b. If I want to retrain, finetune I have to stick to the framework in which the base model was written in

To conclude there are no direct prejudices over tensorflow vs pytorch frameworks. They do the same task with different function names.

If I have the same model available in pytorch and tensorflow, I need to make changes in the model: Many will prefer to use pytorch because of the way in which the forward function is called (to summarize, its easier)

-layer details: different kind of prebuilt layer functions are available for all
- pruning and precision training details are available for all

While we can make any choice when we run in an environment without resource (space, computational complexity) restrictions, what are our choices for resource limited environments?

-onnx: open neural network exchange , it suggests that we can convert any model to this format.i.e. we can convert tensorflow as well as torch models. We can use these on phones, ras-pis, ofcourse jetsons etc

-torchscript: optimizing and minimizing complexity (basically used to simplify torch models)

-tensorrt: a framework introduced by nvidia to optimize tensorflow models i.e. only works with dedicated nvidia hardware(gpu)

Conversion examples:Pytorch to onnx

You can refer [Export a PyTorch model to ONNX — PyTorch Tutorials 2.11.0+cu130 documentation] for more details

```
dummy_x1 = torch.randn(1, 1,32,32)
dummy_x2 = torch.randn(1, 3,60,60)

torch.onnx.export(
    model,
    (dummy_x1, dummy_x2),
    "model.onnx",
    input_names=["input_1", "input_2"],
    output_names=["sum_output", "diff_output"],
dynamic_axes={
        "input_1": {0: "batch_size"},
        "diff_output": {0: "batch_size"}
    },
    opset_version=17
)
```

Convert a regular PyTorch model to **TorchScript** using **JIT** (Just-In-Time compilation).

PyTorch JIT has **two main approaches**:

1. **Tracing** (torch.jit.trace) – good for models with static control flow.
2. **Scripting** (torch.jit.script) – works with dynamic control flow (loops, conditionals).

Example: PyTorch Model → JIT Script & Trace

Python

```
import torch

import torch.nn as nn

import torch.nn.functional as F


# Define a simple model

class SimpleModel(nn.Module):

    def __init__(self):

        super(SimpleModel, self).__init__()

        self.fc1 = nn.Linear(10, 20)

        self.fc2 = nn.Linear(20, 5)
```

```
    def forward(self, x):
        x = F.relu(self.fc1(x))
        return self.fc2(x)

# Create model instance and set to eval mode
model = SimpleModel().eval()

# Example input tensor
example_input = torch.randn(1, 10)

# -------------------------------
# 1. Convert using Scripting
# -------------------------------
try:
    scripted_model = torch.jit.script(model)
    scripted_model.save("model_scripted.pt")
    print("✅ Scripted model saved as model_scripted.pt")
except Exception as e:
    print(f"❌ Scripting failed: {e}")

# -------------------------------
```

```
# 2. Convert using Tracing
# ------------------------------
try:
    traced_model = torch.jit.trace(model, example_input)
    traced_model.save("model_traced.pt")
    print("✅ Traced model saved as model_traced.pt")
except Exception as e:
    print(f"❌ Tracing failed: {e}")

# ------------------------------
# 3. Load and run JIT model
# ------------------------------
try:
    loaded_model = torch.jit.load("model_scripted.pt")
    output = loaded_model(example_input)
    print("Output from loaded scripted model:", output)
except Exception as e:
    print(f"❌ Loading failed: {e}")
```

Key Points:

- **Scripting** analyzes your Python code and converts it to TorchScript, supporting **dynamic control flow**.
- **Tracing** records operations from a sample input — **not suitable** if your model has input-dependent branching.
- Always call .eval() before tracing/scripting for inference models to avoid training-specific behavior.
- Saved .pt files can be loaded in **C++** or deployed without Python.

Config file for onnx

```
name: "object_detection"
platform: "onnxruntime_onnx"
 # Use TensorRT backend if optimized:          "tensorrt_plan"
max_batch_size: 8


input [
  {
    name: "images"          # Must match ONNX model input name
    data_type: TYPE_FP32
    dims: [3, 640, 640]      # Channels, Height, Width
  }
]
```

```
output [
  {
    name: "boxes"            # Bounding boxes: [batch, num_boxes, 4]
    data_type: TYPE_FP32
    dims: [100, 4]
  },
  {
    name: "scores"           # Confidence scores: [batch, num_boxes]
    data_type: TYPE_FP32
    dims: [100]
  },
  {
    name: "labels"           # Class IDs: [batch, num_boxes]
    data_type: TYPE_INT64
    dims: [100]
  }
]

instance_group [
  {
    kind: KIND_GPU
```

```
    count: 1
    gpus: [0]
  }
]

# Optional: dynamic batching for better throughput
dynamic_batching {
  preferred_batch_size: [4, 8]
  max_queue_delay_microseconds: 100
}

# Optional: optimization settings
optimization {
  execution_accelerators {
    gpu_execution_accelerator : [
      { name : "tensorrt" parameters { key: "precision_mode" value:
"FP16" } }
    ]
  }
}
```

Torchscript config

```
name: "my_torchscript_model"
platform: "pytorch_libtorch"
max_batch_size: 8

input [
  {
    name: "INPUT__0"        # Must match TorchScript model input
    data_type: TYPE_FP32
    dims: [ 3, 224, 224 ]   # Example: RGB image
  }
]

output [
  {
    name: "OUTPUT__0"       # Must match TorchScript model
    data_type: TYPE_FP32
    dims: [ 1000 ]          # Example: classification logits
  }
]

instance_group [
  {
    kind: KIND_GPU
    count: 1
  }
]
```

Config for tensorrt

```
name: "tensorrt_detection"
platform: "tensorrt_plan"
max_batch_size: 8

input [
  {
    name: "images"              # Must match the model's input tensor
    data_type: TYPE_FP32        # Change to TYPE_UINT8 if      req
    dims: [3, 640, 640]         # C, H, W
  }
]

output [
  {
    name: "boxes"               # Bounding boxes [batch,
num_detections, 4]
    data_type: TYPE_FP32
    dims: [100, 4]              # Example: top 100 detections
  },
  {
    name: "scores"              # Confidence scores
    data_type: TYPE_FP32
    dims: [100]
  },
  {
    name: "classes"             # Class IDs
    data_type: TYPE_INT64
    dims: [100]
```

```
  }
]

instance_group [
  {
    kind: KIND_GPU
    count: 1
    gpus: [0]                    # GPU ID
  }
]

# Optional: dynamic batching for better throughput
dynamic_batching {
  preferred_batch_size: [4, 8]
  max_queue_delay_microseconds: 100
}

# Optimization settings
optimization {
  execution_accelerators {
    gpu_execution_accelerator : [ {
      name : "tensorrt"
    } ]
  }
}

# Optional: model version policy
version_policy: {
```

```
  latest {
    num_versions: 1
  }
}
```